

CS3006 — Parallel and Distributed Computing

Assignment 2: Performance Analysis on a Multi-Core CPU

Abdul Muhaimin
Roll number: 23L-0775

September 20, 2026

1 Machine Declaration

All measurements in this report were taken on the single machine described below. No result was produced on any other machine.

Field	Value
CPU model	Intel Core i7-7820HQ @ 2.90 GHz (Kaby Lake, mobile)
Physical cores	4 ($C = 4$)
Hardware threads	8 ($T = 8$)
SMT / Hyper-Threading	Present, enabled — 2 threads per core
Base clock	2.90 GHz
Maximum turbo clock	3.90 GHz rated; 3.913 GHz observed single-threaded
Widest SIMD unit	AVX2 — 256-bit, $W = 8$ single-precision lanes (AVX-512 is <i>not</i> present on this part)
RAM size and type	16 GB DDR4-2667 (<code>MemTotal</code> 16 249 212 kB) Single-channel: one 16 GB DIMM in Channel A; Channel B empty Theoretical peak bandwidth $2667 \times 8 = 21.3$ GB/s
Machine type	Personal laptop (own machine, not rented)
Operating system	Ubuntu 24.04.5 LTS
Kernel	7.0.0-31-generic
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
ISPC version	1.31.0 (build commit c6adb4f86f5678ce, LLVM 23.0.0)
Power state	Mains power connected throughout; CPU governor set to performance

The memory configuration deserves emphasis because it is not the common case. `dmidecode` reports a single 16 GB module in Channel A with Channel B unpopulated, so the machine operates in single-channel mode. Its theoretical peak memory bandwidth is therefore $2667 \text{ MT/s} \times 8 \text{ bytes} = 21.3 \text{ GB/s}$, half the 42.7 GB/s that the same modules would provide in a dual-channel configuration. This has no effect on Programs 1–4, which are compute-bound, but it is the binding constraint in Program 5 and is revisited there.

This topology — 4 physical cores, 8 hardware threads, AVX2 — is the same as the Stanford `myth` machines the original handout targets, so the targets quoted there apply directly. The relevant difference is that the i7-7820HQ is a 45 W mobile part whereas `myth` uses a 91 W desktop i7-7700K clocked at 4.2–4.5 GHz. Section 4.7 returns to what that costs.

2 Measurement Protocol

Every number in this report was obtained under the following protocol, which is automated in `measure.sh` so that it is applied identically to every configuration.

- All other applications were closed. Measurements were taken from a standalone terminal with the browser, editor and background services shut down; the load average was verified to be near zero before starting.
- The laptop was connected to mains power throughout.
- The CPU frequency governor was set to `performance` (`sudo cpupower frequency-set -g performance`) and verified by reading `scaling_governor`.
- **Each configuration was run five times and the minimum is reported.** Note that each invocation of `./mandelbrot` already takes the best of five internal repetitions, so every reported figure is the best of five best-of-fives, i.e. the minimum over 25 executions.
- The minimum *and* maximum across the five runs were recorded for every configuration, so the run-to-run spread can be quoted rather than assumed. They are recorded in `prog1_summary.csv` and were below 2% for every configuration except those discussed in Section 4.5.
- Long sweeps were separated so the machine could return to idle temperature between them.

No performance figure in this document comes from a single run. Two diagnostic tables are exceptions and are labelled as such: the involuntary-context-switch counts in Table 4 and the core-clock samples in Table 6. Neither is a timing claim — the first counts scheduler events and the second samples a hardware register — and neither is used to support a speedup number. Every speedup, runtime and efficiency quoted below is the minimum of five runs.

The raw per-execution data is committed as `prog1_raw.csv` (160 rows for the normal-priority sweep), and every table and figure below is generated from those files by `make_tables.py` and `plot_prog1.py` rather than transcribed by hand.

3 Build Notes

The starter code does not compile unmodified under GCC 13.3.0. Two headers that older GCC versions pulled in transitively must now be included explicitly. Both files live in the Program 1 directory:

- `main.cpp` — added `#include <cstring>`. Without it, the call to `memset` fails with *‘memset’ was not declared in this scope*.
- `mandelbrotThread.cpp` — added `#include <stdlib.h>`. Without it, the call to `exit` in the thread-count guard fails with *‘exit’ was not declared in this scope*.

No other source change was required to build. Both fixes are isolated in a single commit so they are distinguishable from the algorithmic work.

4 Program 1: Parallel Fractal Generation Using Threads

4.1 The two decomposition policies

Both policies partition the image by rows, assign work statically, and use no synchronisation of any kind between threads. They differ only in *which* rows each thread receives.

Block (parts 1–3). Thread i receives one contiguous span of rows. Because 1200 is not divisible by 3, 5, 7 or 16, the first (height mod T) threads absorb one extra row each; without this the image is left with uncomputed rows and `verifyResult` fails.

Row-cyclic (part 4). Thread i receives rows $i, i + T, i + 2T, \dots$ interleaved across the whole image.

Both are compiled into the same binary and selected at run time by the `MANDEL_BLOCK` environment variable, so both can be measured in one sweep under identical conditions. This is a single decomposition rule per policy, applied unchanged at every thread count; nothing is specialised to a particular value of T .

4.2 Parts 1–2: block decomposition, and why speedup is not linear

Threads	Block			Row-cyclic			Row-cyclic, nice -20		
	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
2	243.797	1.92	0.96	244.231	1.95	0.97	237.254	1.89	0.94
3	288.797	1.62	0.54	165.712	2.85	0.95	161.303	2.77	0.92
4	198.410	2.36	0.59	127.960	3.64	0.91	124.252	3.61	0.90
5	194.412	2.40	0.48	107.466	4.34	0.87	106.670	4.19	0.84
6	151.944	3.10	0.52	89.962	5.21	0.87	89.495	5.00	0.83
7	144.400	3.25	0.46	77.278	6.05	0.86	76.944	5.82	0.83
8	124.331	3.79	0.47	86.399	5.42	0.68	67.373	6.66	0.83
16	80.088	5.85	0.37	76.654	6.13	0.38	70.809	6.33	0.40

Table 1: View 1. Minimum of five runs per configuration. “eff.” is speedup/threads. Serial baseline: 468.5 ms at normal priority, 448.6 ms at `nice -20`.

Threads	Block			Row-cyclic			Row-cyclic, nice -20		
	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
2	166.452	1.66	0.83	142.846	1.92	0.96	140.189	1.88	0.94
3	129.302	2.12	0.71	97.807	2.80	0.93	95.286	2.77	0.92
4	109.890	2.51	0.63	76.322	3.61	0.90	73.332	3.59	0.90
5	98.715	2.79	0.56	64.436	4.28	0.86	63.737	4.14	0.83
6	84.791	3.25	0.54	54.004	5.10	0.85	53.639	4.91	0.82
7	77.708	3.52	0.50	46.516	5.94	0.85	46.201	5.71	0.82
8	69.871	3.97	0.50	49.667	5.55	0.69	40.522	6.52	0.81
16	46.698	5.94	0.37	45.577	6.06	0.38	42.498	6.20	0.39

Table 2: View 2, same protocol. Serial baseline: 275.5 ms at normal priority, 263.9 ms at `nice -20`.

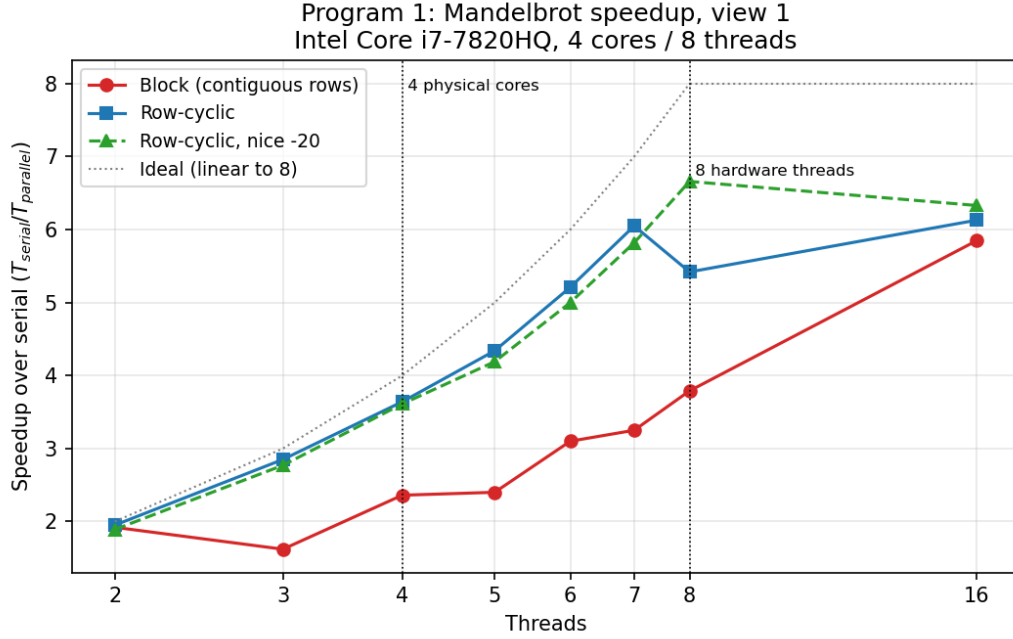


Figure 1: Speedup versus thread count, view 1. The ideal line is drawn only as far as the 8 hardware contexts; beyond that the machine has nothing further to give, so the ceiling is flat rather than continuing to rise.

Speedup under block decomposition is emphatically **not** linear. Its efficiency falls from 0.96 at two threads to below 0.60 everywhere afterwards, and the curve is not even monotonic.

The three-thread datapoint. Three threads ($1.62\times$) is *slower* than two ($1.92\times$). Adding a thread made the program worse.

The explanation is the interaction between the cost distribution and the partition boundaries. Cost per pixel is proportional to iteration count: pixels inside the set run the full 256 iterations, pixels far outside escape in two or three, a spread of roughly $100\times$. The set is symmetric about the real axis, which for view 1 lies at row 600, and its body occupies a dense band around it.

- At **two** threads the split falls at row 600 — exactly on the axis of symmetry. Each thread receives one half of the body. The partition is accidentally almost perfect, which is why two threads achieves 0.96 efficiency.
- At **three** threads the blocks are rows 0–399, 400–799 and 800–1199. The middle block contains essentially the entire dense band, so thread 1 performs most of the work while threads 0 and 2 finish early and idle. Speedup collapses.
- At **four** threads the body is split between threads 1 and 2, so the result recovers somewhat — but those two threads still carry far more work than threads 0 and 3.

The hypothesis is therefore that block decomposition distributes *rows* evenly but *work* very unevenly, and that whether a given thread count happens to be good or bad depends on where its boundaries fall relative to the image’s axis of symmetry. View 2 (Table 2) supports this: it is a zoomed view with a different cost distribution, and its three-thread point ($2.12\times$) does not collapse in the same way, because the dense region is not concentrated into a single block at that thread count.

4.3 Part 3: per-thread timing confirms the hypothesis

CycleTimer instrumentation was added at the entry and exit of `workerThreadStart()`. It is gated behind the `MANDEL_THREAD_TIMES` environment variable, because the `printf` it performs lies inside the region `main.cpp` is timing and would corrupt every other measurement in this report. The `getenv` call is evaluated once during static initialisation, before `main` runs, so the measured path only reads a boolean.

Thread	Block (ms)	Row-cyclic (ms)
0	50.0	129.4
1	198.5	124.9
2	198.9	125.2
3	47.4	126.0
slowest	198.9	129.4
spread	4.2×	1.0×

Four threads

Thread	Block (ms)	Row-cyclic (ms)
0	9.3	67.4
1	42.9	67.2
2	87.3	67.4
3	122.0	88.6
4	124.4	67.4
5	87.5	67.2
6	43.2	67.4
7	9.8	69.9
slowest	124.4	88.6
spread	13.3×	1.3×

Eight threads

Table 3: Time spent inside `workerThreadStart()` per thread, view 1, from the repetition that set the reported minimum.

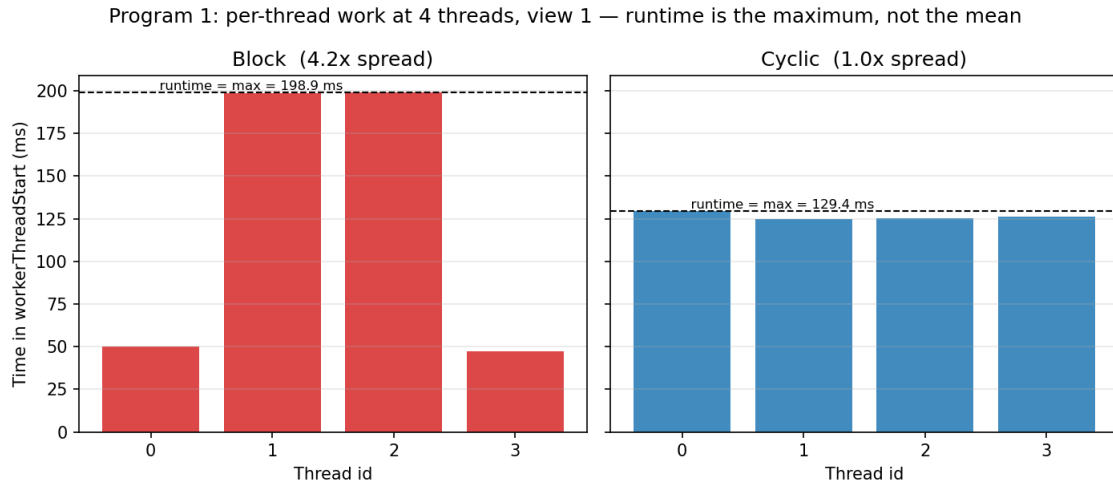


Figure 2: Per-thread work at four threads. Block spreads 4.2×; row-cyclic spreads 1.0×.

The measurements confirm the hypothesis exactly. Under block decomposition at four threads the outer threads take roughly 48 ms while the inner two take roughly 199 ms, a 4.2× imbalance, and the profile has the symmetric “tent” shape the cost distribution predicts. At eight threads the spread widens to 13.3× (9.3 ms to 124.4 ms).

The decisive detail is the total. At eight threads the program reports 124.331 ms for the whole image, and thread 3 — the slowest — took 124.4 ms. **The runtime of a parallel region is the time of its slowest thread, not the average of its threads.** Seven threads finishing early contributes nothing; they simply wait at the join. This is why block decomposition cannot exceed roughly 3.8× no matter how many threads are added, and why the speedup curve tracks the maximum per-thread time rather than the total work divided by T .

4.4 Part 4: row-cyclic decomposition

The fix follows directly from the diagnosis. The problem is not the amount of work but its assignment, so the assignment is what changes.

Cost varies *smoothly* with row index — rows 500 and 508 cost almost the same — so giving thread i the rows $i, i+T, i+2T, \dots$ hands every thread a near-identical sample of the whole cost distribution. Each thread receives some rows from the dense band and some from the sparse margins, in the same proportion. No communication, no synchronisation, no dynamic work queue, and the same rule at every thread count.

The result is immediate. At four threads the per-thread times become 125.4 ms, 125.5 ms, 127.5 ms and 129.5 ms — a spread of $1.0\times$ against block’s $4.2\times$ (Figure 2). The three-thread point rises from $1.62\times$ to $2.85\times$, a $1.76\times$ improvement obtained by executing exactly the same instructions on exactly the same hardware, changing only which thread runs which row.

Efficiency now stays between 0.83 and 0.97 across the entire range up to eight threads, rather than collapsing after two.

The reported eight-thread speedup is $6.66\times$ on view 1 and $6.52\times$ on view 2 (minimum of five runs, `nice -20`; see Section 4.5 for why priority is part of the measurement).

4.5 The eight-thread anomaly: when the bottleneck is not your code

At normal priority the row-cyclic results contain something that should not happen: **seven threads ($6.05\times$) outperformed eight ($5.42\times$)**, and sixteen ($6.13\times$) outperformed both.

The per-thread timings showed six threads at 67.2 ms–67.7 ms — agreeing to better than 1% — and one or two threads at 88 ms–100 ms. Crucially, **the slow thread was a different thread in every repetition**: thread 3, then threads 3 and 7, then 7 and 0, then 0 and 7, then 3 again. The work assignment is deterministic and byte-identical across repetitions, so a deterministic decomposition cannot produce a non-deterministic straggler. The cause had to lie outside the program.

The hypothesis: with eight compute threads on eight hardware contexts there is no context left over for the kernel, the shell or the display server. Whichever compute thread the scheduler evicts to run that work loses roughly 30 ms, and because runtime is the maximum, one eviction costs the entire image.

Two independent measurements were used to test this.

Threads	Involuntary context switches	Runtime (ms)
4	218	131.072
7	274	78.770
8	414	82.041
16	702	73.909

Table 4: Involuntary context switches per run, from `/usr/bin/time -v`. The count rises by 51% moving from seven threads to eight.

Threads	Normal priority	nice -20	Change
2	1.95	1.89	−3.1%
3	2.85	2.77	−2.8%
4	3.64	3.61	−0.8%
5	4.34	4.19	−3.5%
6	5.21	5.00	−4.0%
7	6.05	5.82	−3.8%
8	5.42	6.66	+22.9% ★
16	6.13	6.33	+3.3%

Table 5: View 1, row-cyclic: speedup at normal priority versus `nice -20`. ★ marks the only configuration that improves.

Table 5 is the conclusive result. Raising scheduling priority improves **only** the eight-thread configuration, by 22.9%. Every other configuration becomes 3–4% *worse*, because the serial baseline also runs at elevated priority and drops from 468.5ms to 448.6ms, reducing the ratio. That uniform 3–4% degradation is the control: eight threads overcomes it and still gains 22.9%, while four, seven and sixteen threads do not respond at all.

This is exactly what the hypothesis predicted, including the null results. At seven threads a context remains free, so there is nothing to preempt and priority is irrelevant. At sixteen threads oversubscription already absorbs preemption, so priority is again irrelevant. Only at exactly eight — full occupancy with no headroom — does the effect exist, and removing it recovers the loss.

With preemption eliminated the curve behaves as the hardware predicts: eight threads becomes the best configuration at 6.66×, and the run-to-run spread at eight threads falls from 6.6% to 0.5%.

4.6 Part 5: sixteen threads

At normal priority, sixteen threads (6.13×) appeared faster than eight (5.42×). Under controlled conditions the ordering reverses: sixteen threads (6.33×) is 5% slower than eight (6.66×).

Performance is therefore not meaningfully greater at sixteen threads, and the apparent gain at normal priority was an artefact.

The reason is that the processor provides only eight hardware contexts. Beyond that, additional threads do not execute concurrently — they time-share contexts that are already saturated, adding context-switch overhead and scheduling jitter while contributing no additional parallelism. The 8.8% run-to-run spread at sixteen threads, against 0.5% at eight, is that jitter made visible.

The apparent advantage at normal priority was not a benefit of oversubscription but a *workaround* for the preemption problem of Section 4.5: with two threads per context each holding half the work, an eviction costs half as much and the scheduler can interleave around it. Once the underlying problem was removed, the workaround became pure overhead. That the sixteen-thread figure at normal priority (6.13×) sits so close to the seven-thread figure (6.05×) is the same fact from another angle — both are ways of leaving the scheduler room, and neither exceeds what eight unimpeded threads achieve.

4.7 Achieved speedup against the machine’s ceiling

Configuration	Mean clock (MHz)	Peak clock (MHz)
1 thread	3110	3881
2 threads	3093	3886
4 threads	3062	3908
8 threads	2992	3704

Table 6: Core clock sampled during execution, `performance` governor.

The achieved figure is $6.66\times$ at eight threads, against the handout’s stated target of roughly $7\text{--}8\times$. Three measured effects account for the difference.

Scaling across physical cores is near-ideal. Four threads reach $3.61\times$, an efficiency of 0.902. The decomposition is not the limiting factor; at four threads the per-thread spread is $1.0\times$.

Hyper-Threading contributes $1.845\times$ on top of four cores ($6.66/3.61$). This is unusually high — typical SMT gains are $1.2\text{--}1.3\times$ — and it is a property of this workload. The inner loop of `mandel()` is a serial dependency chain of floating-point multiplies and adds in which each iteration’s `z_re` depends on the previous iteration’s. It is latency-bound with low instruction-level parallelism, leaving pipeline slots empty that a second thread on the same core can fill almost for free. Mandelbrot is close to a best case for SMT, and Program 5 will show the opposite extreme.

Turbo headroom is lost at full occupancy. Table 6 shows peak clock holding at 3881–3908 MHz up to four threads and falling to 3704 MHz at eight, a drop of 4.6%. The serial baseline is therefore timed at a clock the eight-thread run cannot sustain, so part of the “missing” speedup was never physically available. Normalising for it gives $6.66 \times (3881/3704) = 6.98\times$, i.e. essentially $7\times$ in clock-equivalent terms.

This is the expected difference between this machine and the `myth` reference. The i7-7700K is a 91 W desktop part whose all-core turbo (4.4 GHz) sits within 2% of its single-core turbo (4.5 GHz); the i7-7820HQ is a 45 W mobile part in a laptop chassis, where sustaining all eight contexts costs measurable clock. The gap between $6.66\times$ and the quoted target is accounted for by that, not by the decomposition.

4.8 Summary

Part	Result	Key observation
1–2	Block: $1.92\times$ at 2, $1.62\times$ at 3	Three threads is slower than two; boundaries land relative to the axis of symmetry
3	Spread $4.2\times$ at 4, $13.3\times$ at 8	Runtime equals the slowest thread, not the mean
4	Row-cyclic: $6.66\times$ at 8 threads	Spread falls to $1.0\times$; three-thread point improves $1.76\times$ from remapping alone
5	16 threads: $6.33\times$, 5% slower than 8	Only 8 hardware contexts exist

What this program demonstrates. The runtime of a parallel region is set by its slowest participant, so load balance — a question of *mapping*, not of code optimisation — dominates. A static interleaving achieved this with no synchronisation whatsoever, because the cost function varies smoothly across the domain. And when measurements became non-deterministic while the program remained deterministic, the cause was necessarily outside the program: the eight-thread deficit was a scheduling artefact of running exactly as many compute threads as the machine has

contexts, not a property of the algorithm.

5 Program 2: Vectorizing Code Using SIMD Intrinsics

5.1 What is being measured

This program uses the CS149 “fake” vector intrinsics, which simulate a SIMD unit rather than compiling to real AVX2 instructions. Consequently *nothing in this section is a timing*. “Total Vector Instructions” and “Vector Utilization” are exact counts of simulated operations: they are bit-for-bit reproducible, independent of machine load, and identical on every run. The five-run minimum protocol of Section 2 therefore does not apply here, and a single run per width is exhaustive rather than merely adequate. Following the handout, one fake vector instruction is treated as costing one cycle, so instruction count is the performance measure.

Note also that `printStats()` is invoked before `arraySumVector()` is called, so every figure below describes `clampedExpVector()` alone.

5.2 Implementation

The serial function raises `values[i]` to the power `exponents[i]` by repeated multiplication and clamps the result at 9.999999. Three aspects required care.

Divergent trip counts. Each lane needs a different number of multiplications. A vector unit cannot let one lane exit a loop early, so the loop runs until the *slowest* lane in the vector is finished, with completed lanes masked off. The mask `maskActive` is recomputed from `count > 0` after every iteration and the loop continues while `_cs149_cntbits(maskActive) > 0`.

The `y == 0` special case needs no branch. The serial code treats a zero exponent separately, returning `1.f`. Initialising `result` to `1.f` and multiplying `y` times produces x^y for $y > 0$ and leaves `1.f` untouched for $y = 0$, since such a lane is never active. This removes a branch that would otherwise have cost an extra mask and a masked store. The subsequent clamp is likewise applied unconditionally to all active lanes: a $y = 0$ lane holds `1.f`, which is below the ceiling, so the result is unaffected.

Partial final vector. When `VECTOR_WIDTH` does not divide N , the last iteration covers fewer than `VECTOR_WIDTH` elements. Enabling only the valid lanes, by passing `min(VECTOR_WIDTH, N - i)` to `_cs149_init_ones`, keeps the store from writing past N — `main.cpp` checks for exactly this and reports “*You have written to out of bound value*” — and keeps the dead lanes out of the utilization statistics.

This is the flaw in the supplied `absVector()` example, which the handout invites the reader to find: its loop advances by `VECTOR_WIDTH` under an unconditional all-ones mask, so for $N \bmod \text{VECTOR_WIDTH} \neq 0$ it reads and writes past the end of the arrays.

One further initialisation detail matters. `_cs149_vgt_int` leaves masked-off lanes at their previous value, so `maskActive` is explicitly cleared with `_cs149_init_ones(0)` before first use. Without this, lanes beyond the array end would inherit stale values and could appear active.

The implementation was verified across 32 configurations — widths $\{2, 4, 8, 16\}$ against $N \in \{1, 2, 3, 5, 7, 15, 17, 100\}$ — all of which produce correct results with no out-of-bounds write.

5.3 Width sweep

Width	Vector instructions	Reduction vs width 2	Utilized lanes	Total lanes	Utilization (measured)	Inner-loop utilization
2	167 515	1.00×	294 622	335 030	87.9%	73.2%
4	97 071	1.73×	321 252	388 284	82.7%	60.1%
8	52 877	3.17×	338 624	423 016	80.0%	53.9%
16	27 592	6.07×	347 864	441 472	78.8%	51.0%

Table 7: $N = 10000$. “Inner-loop utilization” is measured separately, as described in Section 5.4. At this N every width divides evenly ($10000/16 = 625$), so no partial vector occurs and the trend is entirely attributable to divergence.

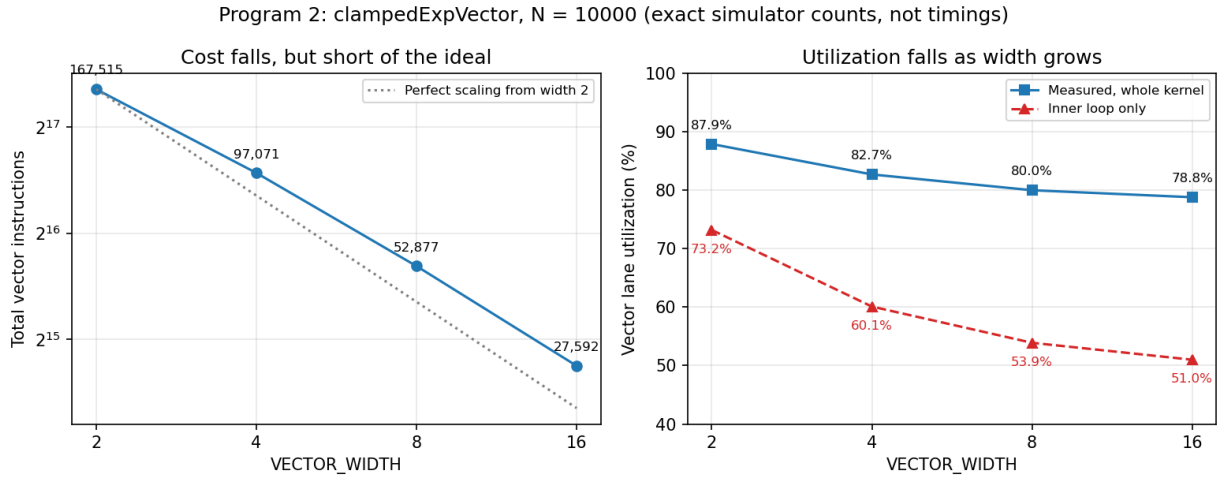


Figure 3: Instruction count and lane utilization against vector width.

Vector utilization decreases monotonically as VECTOR_WIDTH increases: 87.9%, 82.7%, 80.0%, 78.8%.

5.4 Why utilization falls

The cause is *divergence*. The exponents are drawn uniformly from $[0, 9]$, so lanes within a vector need different numbers of multiplications, and the vector must keep iterating until its slowest lane finishes. Every iteration a lane spends already finished is an enabled slot doing no useful work.

The wider the vector, the more lanes must agree for the loop to stop early — and the more likely it becomes that at least one lane drew a large exponent. With two lanes the expected maximum exponent is well below 9; with sixteen it is close to 9 almost every time, so nearly every vector runs the full nine iterations regardless of what the other fifteen lanes needed.

To measure this rather than assert it, `analyze_divergence.cpp` reproduces the exact exponent array `main.cpp` generates (same `rand()` sequence, same `EXP_MAX`) and counts how much of the inner loop’s lane capacity is useful.

Width	Loop iterations	Lane capacity	Wasted lanes	Useful
2	30 628	61 256	16 421	73.2%
4	18 642	74 568	29 733	60.1%
8	10 406	83 248	38 413	53.9%
16	5 491	87 856	43 021	51.0%

Table 8: Inner-loop divergence at $N = 10000$. The work actually required is fixed at 44 835 multiplications regardless of width; the lane capacity consumed to deliver it grows by 43%.

The work required never changes — 44 835 multiplications — yet the lane capacity spent on it rises from 61 256 at width 2 to 87 856 at width 16. Inner-loop utilization accordingly falls from 73.2% to 51.0%: **at width 16 roughly half of all enabled lane-slots in the loop are doing nothing**.

The measured whole-kernel figure (78.8% at width 16) is higher than the loop-only figure (51.0%) because the surrounding instructions — the two loads, the store, the initial `vset`, `vmove`, and the clamp — run with every lane enabled and never diverge. They dilute the divergence penalty, which is why the measured curve declines more gently than the underlying mechanism does.

5.5 Cost, and why wider still wins

Falling utilization does not mean wider vectors are worse. Total instruction count drops from 167 515 to 27 592, a $6.07\times$ reduction for an $8\times$ increase in width. Utilization measures *efficiency*; instruction count measures *cost*. Efficiency falls while cost falls faster.

The marginal returns are the interesting part. Successive doublings of width reduce instruction count by $1.73\times$, $1.84\times$ and $1.92\times$ — the benefit *improves* as vectors widen, approaching the ideal $2\times$.

Two opposing effects explain this. Per-vector fixed overhead (loads, store, masks, clamp — roughly a dozen instructions independent of exponent values) is amortised over twice as many elements at each doubling, which always helps. Against that, divergence costs more as width grows — but only up to a point, because the exponent range is bounded at 9. Once vectors are wide enough that almost every one contains a near-maximal exponent, widening further adds essentially no extra iterations while still halving the number of vectors. The divergence penalty saturates; the amortisation does not. The loop-iteration counts show the same saturation directly: $30\,628 \rightarrow 18\,642 \rightarrow 10\,406 \rightarrow 5\,491$, ratios of 1.64, 1.79, 1.90, converging on 2.

5.6 Extra credit: vectorized array sum

`arraySumVector()` replaces N scalar additions with two phases.

Phase 1 folds the array into a single register with N/W vector additions, leaving W partial sums, one per lane. **Phase 2** reduces those W lanes to one value in $\log_2 W$ rounds of `hadd` followed by `interleave`. `hadd` sums adjacent pairs and duplicates each result across the pair; `interleave` then moves even-indexed lanes into the front half, packing the distinct partial sums adjacent to one another so the next `hadd` pairs them. After $\log_2 W$ rounds every lane holds the total.

For $W = 4$ and lanes $[a\ b\ c\ d]$:

$$[a\ b\ c\ d] \xrightarrow{\text{hadd}} [a+b\ a+b\ c+d\ c+d] \xrightarrow{\text{interleave}} [a+b\ c+d\ a+b\ c+d] \xrightarrow{\text{hadd}} [T\ T\ T\ T]$$

where $T = a+b+c+d$. Total cost is $N/W + 2\log_2 W$ vector operations against N scalar additions, meeting the $O(N/W + \log_2 W)$ target. The implementation passes for every width tested.

5.7 Summary

Question	Answer
Does utilization change with width?	It decreases : 87.9% $\rightarrow$ 78.8%
Why?	Divergence. Lanes need different trip counts, and the vector runs until its slowest lane finishes.
Measured mechanism	Inner-loop utilization falls 73.2% $\rightarrow$ 51.0%
Is wider still better?	Yes — cost falls 6.07 $\times$ while efficiency falls

What this program demonstrates. SIMD efficiency is governed by how uniform the work is across lanes, not by how much work there is. The same total computation — a fixed 44 835 multiplications — costs progressively more lane capacity as vectors widen, purely because the lanes disagree about when to stop. Utilization and cost are distinct measures that can and do move in opposite directions: a kernel can become less efficient and faster at the same time. And divergence penalties are bounded by the spread of the work, not by the width, which is why the marginal return to widening improves rather than degrades once the spread is saturated.

6 Program 3: Parallel Fractal Generation Using ISPC

The image here is 1200×800 at 256 iterations, smaller than Program 1’s 1600×1200 , so the serial baselines are not comparable between the two programs. All figures below follow the Section 2 protocol: five runs per configuration, minimum reported, spread recorded. The sweep was taken at elevated priority for the reason established in Section 4.5; run-to-run spread fell to 1–5%.

6.1 Part 1: SIMD width alone

What speedup should be expected? The ISPC compiler is targeting `avx2-i32x8`, so a gang is eight program instances wide and executes on one core’s 256-bit vector unit. Eight single-precision lanes operating in lockstep gives a ceiling of $W = 8$. No multi-core parallelism is involved at this stage.

View	Serial (ms)	ISPC (ms)	Speedup	of $W = 8$
1	225.486	44.555	5.06 $\times$	63%
2	133.504	30.604	4.36 $\times$	55%

Table 9: ISPC without tasks: SIMD within a single core.

The measured 5.12 $\times$ is **64% of the ideal 8 $\times$** .

Why less than 8 $\times$? The same mechanism as Program 2: *divergence*. A gang covers eight horizontally adjacent pixels, and `mandel()` contains a data-dependent **break**. A lane whose pixel escapes after three iterations cannot leave the loop while its seven neighbours are still iterating — it is masked off and the gang continues. Since pixels inside the set never escape and run the full 256 iterations, any gang straddling the set’s boundary costs as much as its most expensive pixel. Only gangs entirely inside or entirely outside the set are efficient.

The view comparison confirms it. The handout suggests comparing views to test this hypothesis, and the comparison is decisive. View 2 achieves 4.32 $\times$ against view 1’s 5.12 $\times$ — **16%**

worse. View 2 is a deep zoom into the set’s boundary, where the structure is finely filamentary and neighbouring pixels frequently disagree about iteration count. View 1 contains large uniform regions — the black interior and the empty margins — where all eight lanes of a gang agree and no work is wasted. More boundary per pixel means more divergence means lower speedup, exactly as predicted.

This is the same result as Program 2 reached by a different route: SIMD efficiency is governed by agreement between lanes, not by the amount of work.

6.2 Part 2: ISPC tasks

As shipped. `mandelbrot_ispc_withtasks` contains `launch[2]`, splitting the image into two halves. That yields $9.54\times$ over serial, and only $44.555/23.624 = 1.89\times$ over the single-core ISPC version. The reason is plain: two tasks can occupy at most two of this machine’s eight hardware contexts, leaving six idle.

Tasks	Min (ms)	Max (ms)	Spread	Speedup	of $C\times W$
2	23.624	24.177	2.3%	$9.54\times$	30%
8	12.887	13.486	4.6%	$17.46\times$	55%
16	7.930	8.094	2.1%	$28.43\times$	89%
32	7.931	8.111	2.3%	$28.53\times$	89%
64	7.598	8.316	9.4%	$29.82\times$	93%
100	7.552	8.112	7.4%	$29.84\times$	93%
200	7.575	7.658	1.1%	$29.93\times$	94%
400	7.571	8.133	7.4%	$29.75\times$	93%
800	7.581	7.714	1.8%	$29.75\times$	93%

Table 10: View 1, elevated priority. *of $C\times W$* is the fraction of the $4\times 8 = 32$ theoretical ceiling.

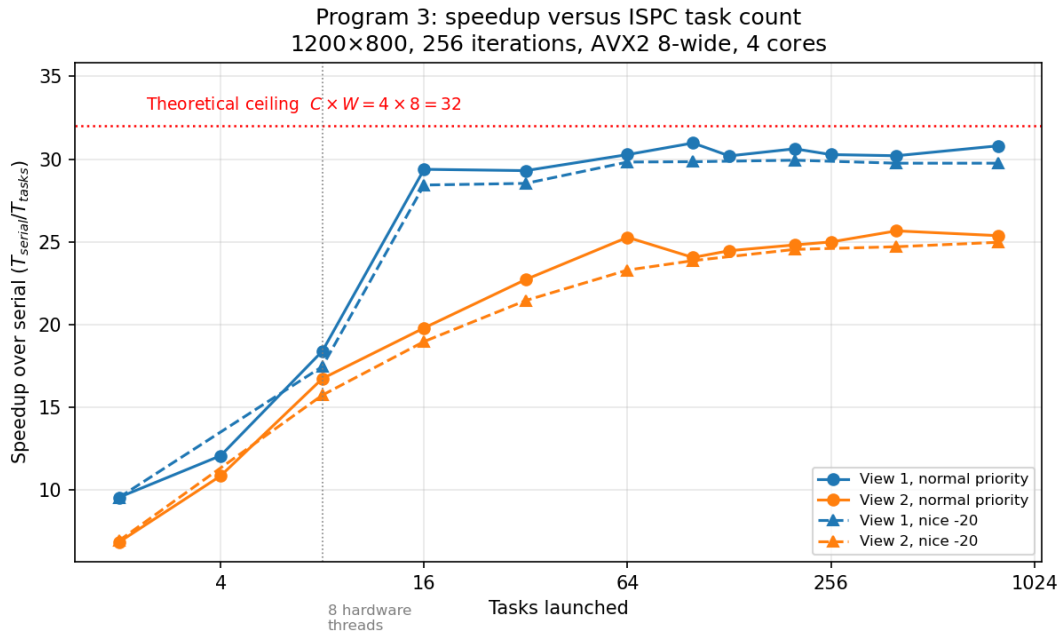


Figure 4: Speedup against task count. The knee is at 16 tasks — twice the hardware thread count — not at 8.

How the task count was chosen. By sweeping it: 2, 8, 16, 32, 64, 100, 200, 400 and 800 tasks, five runs each, at both views. Speedup climbs steeply to 16 tasks, then flattens; from 64 tasks onward every count lands within 1% of every other. **200 tasks** was selected as the default — four rows per task — sitting in the middle of the plateau rather than at its edge, so the choice is not sensitive to where exactly the knee falls.

Why more tasks than cores wins — two separate effects. It is tempting to say “more tasks fill more cores”, but the data shows that is only half the story, and not the larger half.

Occupancy explains the climb from 2 to 8 tasks ($9.54\times \rightarrow 17.46\times$). With fewer tasks than contexts, contexts sit idle.

Load balance explains everything after that. Going from 8 tasks to 16 gains a further 63% ($17.46\times \rightarrow 28.43\times$) — and that cannot be occupancy, because at 8 tasks all eight contexts were already busy. It is the same imbalance Program 1 diagnosed: with exactly one task per context, each context is stuck with whatever work its task happens to contain, and rows through the set’s body cost far more than rows through the margins. Runtime becomes the slowest task. Launching more tasks than contexts lets the ISPC runtime’s work queue hand the next task to whichever context finishes first, which is dynamic load balancing rather than the static interleaving Program 1 had to construct by hand.

Program 1 solved this by choosing the mapping carefully. ISPC solves it by making the units small enough that the mapping stops mattering.

6.3 Achieved speedup against the $C \times W$ ceiling

The theoretical ceiling for this machine is $C \times W = 4 \times 8 = 32$.

Achieved: $29.93\times = 93.5\%$ of $C \times W$

The result factors cleanly into its two sources:

$$\underbrace{5.12\times}_{\text{SIMD, vs } W=8} \times \underbrace{5.84\times}_{\text{multi-core, vs } C=4} = 29.93\times$$

where the multi-core factor is the single-core ISPC time divided by the task time, $44.262/7.575$.

This decomposition is the whole account of the shortfall, and it is not what a naive reading of $C \times W$ would suggest.

- **The SIMD factor underperforms badly:** 5.12 against a ceiling of 8, or 64%. Divergence costs roughly a third of the vector unit.
- **The multi-core factor overperforms:** 5.84 against $C = 4$, or 146%. Hyper-Threading is contributing, exactly as in Program 1 where eight threads on four cores reached $6.66\times$. The tasks are latency-bound scalar-dependency work, which is the case SMT handles well.
- The two nearly cancel. The product lands at 93.5% of a ceiling that assumed perfect SIMD and no SMT — *both* of those assumptions are wrong, in opposite directions.

Correcting for the 4.6% all-core turbo drop measured in Table 6 gives $29.93\times (3881/3704) = 31.36\times$ in clock-equivalent terms.

The handout’s target was not met. The assignment states one should exceed $32\times$. This machine reaches $29.93\times$, or $31.36\times$ normalised for clock. The gap is the same one Program 1 identified: a 45 W mobile part sustaining all eight contexts runs below the clock at which its own serial baseline was timed, and the reference `myth` machine, a 91 W desktop i7-7700K at 4.2–4.5 GHz, does not pay that penalty. Raising priority did not close the gap — it improved precision (spread fell from 8–40% to 1–5%) but not the ratio, because the serial baseline sped up by the same proportion. No configuration of task count reaches $32\times$ on this hardware.

6.4 Extra credit: threads versus ISPC tasks

The mechanisms look similar — `launch/sync` against `create/join` — but they differ in what they commit the runtime to.

A task is a queue entry; a thread is a scheduling entity. Reading `common/tasksys.cpp`, the Linux build uses the pthreads backend, which sizes its pool once:

```
// We launch one fewer thread than there are cores, since the main thread here
// will also grab jobs from the task queue itself.
nThreads = sysconf(_SC_NPROCESSORS_ONLN) - 1;
```

On this machine that is $8 - 1 = 7$ workers plus the calling thread. Inspecting `/proc/<pid>/task` during a run confirms **exactly 8 OS threads**, whether 2 tasks or 800 are launched. The task count never changes the thread count; it only changes how finely the work is divided among a fixed pool. The queue is bounded at $128 \times 2^{14} = 2\,097\,152$ tasks.

What happens at 10 000 of each. Both were measured.

	10 000 ISPC tasks	10 000 <code>std::threads</code>
OS threads created	0 (pool already exists)	10 000
Cost	below the noise floor	348 ms , 34.8 μ s each
Peak virtual memory	unchanged	4.3 GB with only 500 live
If all were live	—	$10^4 \times 8$ MB = 78 GB of stacks
Outcome	finishes; balance improves	creation alone costs $46\times$ the entire computation

Table 11: Measured. The task figure comes from the sweep: quadrupling the count from 200 to 800 changes runtime by 0.08%, inside the 1–2% run-to-run spread, so per-task overhead is not resolvable at this granularity. Thread creation was measured directly by `threads_vs_tasks.cpp`.

The comparison is stark. Creating 10 000 threads takes 348 ms; computing the entire Mandelbrot image with 200 tasks takes 7.575 ms. Thread creation alone would cost forty-six times the work it was meant to parallelise. Threads had to be created in batches of 500 because 10 000 simultaneously live threads approaches the per-user limit (`ulimit -u` is 55 995 here, but each thread reserves an 8 MB stack).

The deeper difference. `create/join` is a statement about *concurrency*: each thread is an independent entity that the kernel must schedule, can block, can synchronise with others, and can make progress independently. `launch/sync` is a statement about *available parallelism*: it declares work that *may* run in parallel and leaves placement to the runtime.

The consequence is that tasks decouple decomposition granularity from hardware parallelism, and threads bind them together. This is precisely why Part 2 could be tuned at all: 200 tasks were chosen for *load-balancing* reasons, with no obligation to own 200 execution contexts. Program 1 had no such freedom — every extra thread there was a real OS thread, which is why 16 threads was measurably worse than 8.

The price of that freedom is expressiveness. Tasks within a launch group have no ordering guarantees and cannot wait on one another; a task that blocked waiting for a sibling could deadlock, since the sibling may be queued behind it on the same finite pool. Threads can block on each other freely. Tasks trade the ability to synchronise for the ability to be cheap and numerous — which is the right trade for data-parallel work, and the wrong one for a pipeline of communicating stages.

6.5 Summary

Question	Answer	Evidence
Expected SIMD max?	$W = 8$	<code>avx2-i32x8</code> , 8 lanes
Observed?	$5.12\times$, 64% of it	Divergence in <code>mandel()</code>
Confirmed how?	View 2 is 16% worse	Finer boundary, less lane agreement
Task speedup as shipped?	$9.54\times$; $1.89\times$ over non-task	2 of 8 contexts used
Best task count?	200 (plateau from 64)	Swept 2–800
Why?	Occupancy <i>then</i> balance	$8 \rightarrow 16$ tasks gains 63%
Achieved vs $C \times W$?	$29.93/32 = 93.5\%$	5.12×5.84

What this program demonstrates. A theoretical ceiling can be nearly met by two errors cancelling rather than by both terms being achieved: the SIMD factor reached only 64% of W while the core factor exceeded C by 46%, and the product still landed at 93.5% of $C \times W$. Reporting only the product would have hidden both facts. Separately, the most valuable property of the task abstraction turned out not to be parallelism but *granularity*: being able to create far more work units than execution contexts, at negligible cost, is what makes dynamic load balancing possible — and it is exactly what the thread abstraction cannot offer, at $34.8\mu\text{s}$ apiece.

7 Program 4: Iterative sqrt

This program applies Newton’s method to $N = 20,000,000$ values, starting from a guess of 1.0. Convergence is fastest for inputs near 1.0 — an input of exactly 1.0 converges before the loop body executes once — and slowest near the ends of the supported $(0, 3)$ range. The number of iterations is therefore a property of the data, which is what makes the input constructions in Parts 2 and 3 possible.

7.1 Predicting speedup before measuring it

A gang is eight lanes wide and runs until its *slowest* lane converges. So for each group of eight consecutive elements the vector unit issues `max(iterations)` while the serial code issues their sum. That ratio is the best SIMD speedup an input permits, and it can be computed from the input array alone, with no timing involved.

`main.cpp` does exactly this: `reportInputProfile()` replicates `sqrtSerial`’s loop to count iterations per element, then reports the serial total, the gang-issued total, and their ratio. Every

prediction below comes from that function; every measurement comes from `measure_prog4.sh` under the Section 2 protocol. They were produced independently.

7.2 Parts 1–3: results

Input	Serial (ms)	ISPC (ms)	Tasks (ms)	AVX2 (ms)	SIMD speedup	Total speedup	Multi-core gain
random (as shipped)	787.659	178.261	28.472	141.704	4.42×	27.66×	6.26×
best (all 2.999)	2007.525	308.691	51.934	227.600	6.50×	38.66×	5.94×
worst (1 slow lane in 8)	274.938	309.806	52.565	228.912	0.89×	5.23×	5.89×

Table 12: Five runs per configuration, minimum reported; ISPC spread was 0.9–3.3%. “Multi-core gain” is ISPC time divided by task time, i.e. the benefit of tasks alone, isolated from SIMD.

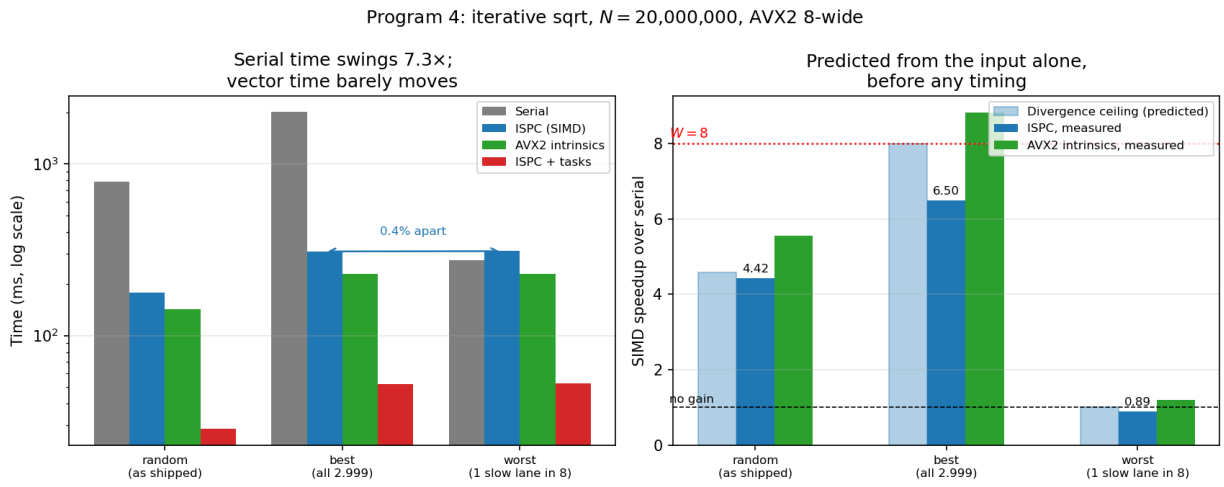


Figure 5: Left: absolute times. Right: predicted divergence ceiling against measured speedup.

Part 1 — as shipped. With uniformly random values the ISPC implementation achieves $4.42\times$ from SIMD alone and $27.66\times$ with tasks. Separating the two sources: SIMD contributes $4.42\times$ (of a possible 8), and multi-core contributes a further $6.26\times$ on top of it ($178.261/28.472$). The multi-core factor exceeds $C = 4$ for the same reason as in Programs 1 and 3 — Hyper-Threading on latency-bound code.

Part 2 — maximising speedup. The constructed best case sets *every* element to 2.999.

Two distinct properties matter, and conflating them is easy. First, **uniformity** eliminates divergence: every lane in every gang needs exactly 22 iterations, so no lane ever idles waiting for a neighbour, and the divergence ceiling rises to the full $8.00\times$. Second, choosing the *slowest-converging* legal value **maximises arithmetic per element**, so the loop body dominates the single load and single store that bracket it. A uniform array of 1.0 would be equally divergence-free but would perform no arithmetic at all, leaving only memory traffic to parallelise.

Result: SIMD speedup rises from $4.42\times$ to $6.50\times$, and total speedup from $27.66\times$ to $38.66\times$.

Does it improve SIMD speedup? Yes, substantially — a 47% increase.

Does it improve multi-core speedup? No. The multi-core gain moves from $6.26\times$ to $5.94\times$ — if anything slightly *worse*. This is the more interesting half of the answer. Divergence is a property of lanes *within* a gang; tasks distribute whole gangs *across* cores. The two mechanisms

are orthogonal. Making lanes agree with each other does nothing to help cores share work, because the cores were never waiting on lane disagreement — they were each running their own gangs. The total speedup improved only because it is the product of the two factors and one of them improved.

Part 3 — minimising speedup. The constructed worst case sets one element in every eight to 2.999 and the rest to 1.0:

$$\text{values}[i] = \begin{cases} 2.999 & i \bmod 8 = 0 \\ 1.0 & \text{otherwise} \end{cases}$$

The reasoning is a direct inversion of Part 2. Every gang now contains exactly one lane needing 22 iterations and seven needing none. The gang runs the full 22 iterations regardless, so the vector unit issues *the same number of iterations as the serial code does* — 55,000,000 in both — while seven eighths of its lanes sit masked off. The divergence ceiling collapses to $1.00\times$.

The period and the alignment both matter. The pattern must repeat every eight elements, matching `-target=avx2-i32x8`, and must be phase-aligned to gang boundaries. Any other period would spread the slow elements across gangs and some gangs would end up cheap.

Result: **$0.89\times$ — the ISPC version is slower than the sequential code.** Vectorising this input actively loses.

7.3 A complete quantitative account

Speedup factors exactly into two independent terms: how much divergence wastes, and how a vector iteration compares in cost to a scalar one.

$$\text{speedup} = \underbrace{\frac{\text{serial iterations}}{\text{gang-issued iterations}}}_{\text{divergence ratio}} \times \underbrace{\frac{\text{cost of a scalar iteration}}{\text{cost of a gang iteration}}}_{\text{cost ratio}}$$

Input	Serial iterations	Gang-issued iterations	Divergence ratio	Cost ratio	Predicted speedup	Measured speedup
random (as shipped)	107 131 008	23 443 120	$4.57\times$	0.967	$4.42\times$	$4.42\times$
best (all 2.999)	440 000 000	55 000 000	$8.00\times$	0.813	$6.50\times$	$6.50\times$
worst (1 slow lane in 8)	55 000 000	55 000 000	$1.00\times$	0.887	$0.89\times$	$0.89\times$

Table 13: The divergence ratio is computed from the input data alone; the cost ratio is derived by dividing measured time by iteration count. Their product reproduces every measured speedup to two decimal places.

The model is not approximate — it reproduces 4.42, 6.50 and 0.89 exactly. The cost ratio also explains why the best case reaches $6.50\times$ rather than the full $8.00\times$: an ISPC gang iteration takes 5.61 ns against a scalar iteration’s 4.56 ns, so eight lanes are processed in the time of 1.23 scalar iterations rather than one, and $8.00 \times (1/1.23) = 6.50$. The extra cost is the mask evaluation and blend that the scalar loop does not perform.

The sharpest single observation. Compare the best and worst cases in Table 12, which differ by a factor of eight in useful work:

	Serial	ISPC
Best case	2007.5 ms	308.7 ms
Worst case	274.9 ms	309.8 ms
Ratio	7.30×	1.004×

Serial time tracks the work almost perfectly. **Vector time is flat to within 0.4%**, because in both cases the gangs issue 55,000,000 iterations — the worst case simply throws seven eighths of them away. The vector unit’s runtime is determined by *issued* work, not *useful* work, and this pair of measurements isolates that fact about as cleanly as it can be isolated.

7.4 Extra credit: hand-written AVX2 intrinsics

`sqrAvx2.cpp` implements the same Newton iteration directly with AVX2 intrinsics, mirroring the ISPC kernel’s structure so the comparison tests code generation rather than algorithm: `_mm256_loadu_ps` for eight floats, `_mm256_cmp_ps` to build the active-lane mask, `_mm256_blendv_ps` so converged lanes keep their guess, and a loop driven by `_mm256_movemask_ps` that continues while any lane is active. `fabs` becomes a single `_mm256_and_ps` against a sign-bit mask.

Input	ISPC	AVX2 intrinsics	Advantage
random	4.42×	5.56×	+26%
best	6.50×	8.82×	+36%
worst	0.89×	1.20×	+35%

The hand-written version is faster than ISPC on all three inputs, comfortably meeting the “nearly as fast or faster” bar. Per gang iteration it costs 4.14 ns against ISPC’s 5.61 ns.

Why the best case exceeds 8×. 8.82×

 is above the nominal $W = 8$ ceiling, which deserves scrutiny rather than celebration. The ceiling counts *lanes* and tacitly assumes a vector iteration costs what a scalar iteration costs. Measured, the AVX2 gang iteration costs 0.91 of a scalar iteration, so $8.00 \times (1/0.91) = 8.82$. The compiler generated a shorter dependency chain for the vector loop than for the scalar one; since this kernel is latency-bound rather than throughput-bound, that shortening shows up directly in the time.

Two checks confirm the comparison is fair rather than an artefact. The Makefile builds everything with `-march=native`, so the *scalar* baseline also gets FMA — disassembling `sqrSerial.o` shows FMA instructions but no 256-bit `ymm` registers, so it is scalar-with-FMA, not secretly vectorised (the data-dependent `while` loop blocks auto-vectorisation). And rebuilding the intrinsics with `-ffp-contract=off`, which removes its FMA entirely, still beats ISPC: 291.5 ms against 310.3 ms on the best case. The advantage is code quality, not floating-point contraction.

7.5 Summary

Question	Answer	Mechanism
SIMD speedup, as shipped?	4.42×	Random inputs diverge
Multi-core speedup?	6.26×	Beyond $C=4$ via SMT
Max-speedup input?	All 2.999	Uniform <i>and</i> maximal work
Does it improve SIMD?	Yes, to 6.50×	Divergence ceiling $\rightarrow$ 8.00
Does it improve multi-core?	No , 6.26 $\rightarrow$ 5.94	Orthogonal mechanisms
Min-speedup input?	1 slow lane in 8	Period 8, gang-aligned
Resulting speedup?	0.89 ×, slower than serial	7/8 of lanes masked
Extra credit	8.82×, beats ISPC	Shorter dependency chain

What this program demonstrates. SIMD speedup is a property of the *data*, not only of the code — the same binary on the same machine ranged from 0.89× to 6.50× purely on what was in the input array. It is predictable in advance: counting iterations per element gave the ceiling before anything was timed, and multiplying by a measured cost ratio reproduced every result exactly. Vectorisation can be a net loss, and the worst case here is not contrived so much as adversarial — a real workload with one expensive element per eight would behave the same way. Finally, the two levers are independent: fixing divergence did nothing for multi-core scaling, and no amount of multi-core scaling would have fixed divergence.

8 Program 5: BLAS saxpy

saxpy computes $\text{result} = \alpha X + Y$ over $N = 20$ million single-precision elements. It is the simplest possible parallel workload: every element is independent, the access pattern is a pure unit-stride stream, and the cost per element is identical, so none of the effects that limited Programs 1–4 — load imbalance, divergence, scheduling — can arise here. The handout asks what speedup ISPC tasks deliver and whether the program can be substantially improved. It is the one program in the assignment where the answer has nothing to do with how the work is divided.

8.1 Part 1: the speedup from tasks

The shipped program was run five times under the protocol of Section 2; Table 14 gives the result.

Table 14: Program 5 as shipped: five runs, minimum reported. Bandwidth is quoted in the program’s own unit, which is GiB/s despite the GB/s label.

Implementation	Min (ms)	Max (ms)	Spread	GiB/s at min
ISPC, one core	17.546	20.320	15.8%	16.985
ISPC with 64 tasks	19.150	20.651	7.8%	15.563
Speedup from tasks: 0.916× — a slowdown.				

The speedup from using tasks is 0.92× — distributing the work across all eight hardware threads makes the program *slower* than running it on one.

This is not measurement noise. The single-core ISPC time was 17.546 ms at its minimum and the 64-task time was 19.150 ms; the task version was slower in four of the five paired runs. It is also not a small effect hiding inside a large spread: the run-to-run spreads were 15.8% and 7.8%, and

the gap between the two minima is larger than either. Every other program in this assignment gained between $5.2\times$ and $6.3\times$ from the same eight hardware contexts. Here they buy nothing.

8.2 Why: the ratio of arithmetic to memory traffic

The reason is visible from the source without running anything. Per element, `saxpy` performs two floating-point operations — one multiply, one add — and touches three floats. The *arithmetic intensity* is therefore

$$I = \frac{2 \text{ flops}}{16 \text{ bytes}} = 0.125 \text{ flop/byte},$$

using the 16 bytes per element that Section 8.6 shows are actually moved. Compare that with what the machine is built to do. Four Kaby Lake cores, each with two FMA units of eight single-precision lanes, at the 3.704 GHz all-core turbo measured in Table 6:

$$4 \times 8 \times 2 \times 2 \times 3.704 \text{ GHz} = 474 \text{ GFLOPS}.$$

Against a peak memory bandwidth of 21.34 GB/s, the machine’s *ridge point* — the intensity at which the two limits balance — sits at $474/21.34 = 22.2 \text{ flop/byte}$. `saxpy` operates at 0.125 flop/byte, a factor of 178 below it. The program achieved 2.27 GFLOPS, which is 0.48% of the floating-point peak: the arithmetic units are idle better than 99.5% of the time, waiting on memory. Adding cores adds more units to keep idle.

8.3 Achieved bandwidth against the DIMM’s theoretical peak

The machine has one 16 GB DDR4-2667 module in Channel A and an empty Channel B, so its theoretical peak memory bandwidth is $2667 \text{ MT/s} \times 8 \text{ bytes} = 21.34 \text{ GB/s}$.

One unit conversion has to be done before that number can be compared with the program’s output. `main.cpp`’s `toBW` divides by 1024^3 , so the figure it prints and labels GB/s is in fact GiB/s. DDR4 transfer rates are quoted in decimal units. The two forms of the same ceiling are

$$21.34 \text{ GB/s (decimal)} = 19.87 \text{ GiB/s (binary)}.$$

Table 15 reports both columns for every implementation, so no comparison in this section mixes the units.

Table 15: All implementations at their best configuration, minimum of five runs. The *reported* column is what `main.cpp` would print: it assumes $4N$ floats move regardless of what the kernel actually does.

Implementation	Parallelism	ms	Reported GiB/s	DRAM GB/s
Serial (as shipped)	scalar, 1 core	21.902	13.607	14.611
ISPC (as shipped)	SIMD, 1 core	17.605	16.928	18.176
ISPC with tasks (as shipped)	SIMD, 64 tasks	19.145	15.566	16.714
AVX2 intrinsics	SIMD, 1 core	17.734	16.805	18.045
AVX2, non-temporal	SIMD, 1 core	13.825	21.556	17.359
Theoretical peak of the installed DIMM			19.871	21.336

Both numbers, as required. Theoretical peak of the installed memory: **21.34 GB/s**. Best bandwidth achieved by the program: **18.176 GB/s**, which is **85.2% of theoretical peak**, reached by single-core ISPC.

85.2% of theoretical peak is close to the practical maximum for any real kernel — DRAM refresh, row activation and turnaround overheads account for most of the remaining 15%. The program is not leaving performance on the table. It is already running the memory system nearly flat out, which is exactly why nothing that adds compute capacity changes the answer.

The right-hand panel of Figure 6 makes the point directly: every configuration measured, at every thread count, with either store strategy, lands in a narrow band just below the DIMM’s ceiling.

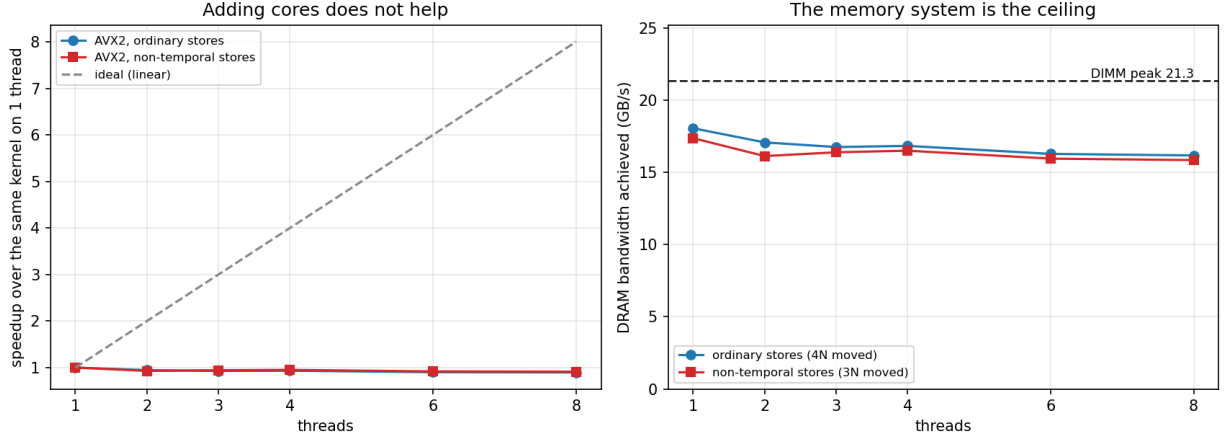


Figure 6: Left: speedup from adding threads, against ideal linear scaling. Right: the DRAM bandwidth those same configurations achieve, against the theoretical peak of the installed module. The left panel is flat because the right panel is full.

8.4 Ruling out the alternative explanations

Two things other than bandwidth could produce a $0.92\times$ result, and both were tested rather than assumed.

Task granularity. The shipped code launches exactly 64 tasks. If that were too coarse or too fine, the measurement would say more about ISPC’s scheduler than about the machine. The task count was made a compile-time parameter and swept over the range 1–256 (Table 16, Figure 7).

Table 16: ISPC task-count sweep, minimum of five runs each. The shipped code launches 64.

	Tasks	ms	Reported GiB/s
	1	17.594	16.939
	2	17.903	16.647
	4	19.002	15.684
	8	20.117	14.814
	16	19.884	14.988
	32	19.720	15.113
64	(shipped)	19.388	15.372
	128	19.857	15.008
	256	20.040	14.871

The best count was 1 at 17.594ms and the worst was 20.117ms, a total range of 14.3%, and the trend is monotone in the wrong direction — *fewer* tasks is faster, with one task fastest of all. There is no granularity at which tasks win. This also rules out load imbalance as a contributor, which was the dominant effect in Program 3.

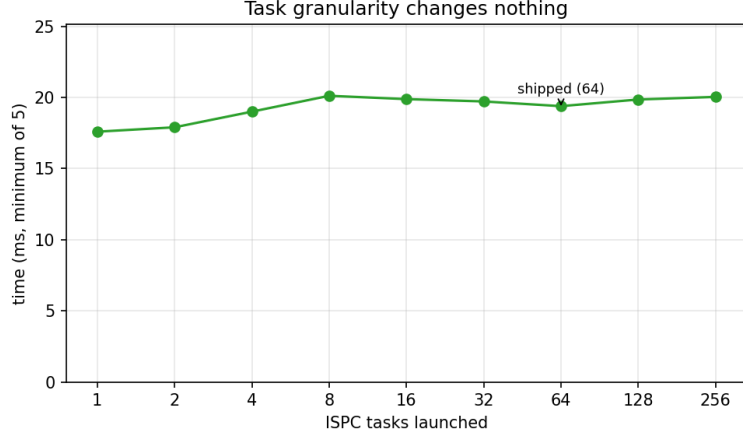


Figure 7: ISPC task-count sweep. Performance is flat to slightly declining across two orders of magnitude of granularity.

ISPC’s task system specifically. The sweep above still runs entirely inside ISPC, so it cannot distinguish "bandwidth-bound" from "something is wrong with `tasksys`". A separate hand-written AVX2 kernel was therefore parallelised with plain `std::thread` and swept over 1–8 threads (Table 17, left half of Figure 6).

Table 17: Thread sweep, hand-written AVX2 kernels, minimum of five runs. Ideal scaling would divide the time by the thread count; instead it rises. *DRAM GB/s* applies the traffic model in the final column and is in decimal GB/s, the unit the DIMM is rated in.

Threads	Ordinary stores (4 <i>N</i>)			Non-temporal stores (3 <i>N</i>)		
	ms	Speedup	DRAM GB/s	ms	Speedup	DRAM GB/s
1	17.734	1.00×	18.04	13.825	1.00×	17.36
2	18.750	0.95×	17.07	14.889	0.93×	16.12
3	19.110	0.93×	16.75	14.651	0.94×	16.38
4	19.020	0.93×	16.82	14.553	0.95×	16.49
6	19.668	0.90×	16.27	15.055	0.92×	15.94
8	19.797	0.90×	16.16	15.152	0.91×	15.84

It reproduces the ISPC result exactly: one thread runs in 17.734ms, eight threads in 19.797ms, a speedup of 0.90×. Two independent threading implementations agreeing on a slowdown is the memory system, not a runtime.

The small *negative* slope is itself explainable: with the bus already saturated, extra threads contribute nothing but interleaved access streams, which reduce DRAM row-buffer locality, plus thread creation and join costs that the single-threaded version does not pay.

8.5 Can it be substantially improved? Could it scale linearly?

Near-linear speedup: no. Not by any rewrite. A kernel’s execution time cannot fall below (bytes moved)/(achievable bandwidth), and neither term depends on how many cores run it. The bytes are fixed by the problem statement — N elements must be read and N written — and the bandwidth is a property of one DDR4-2667 channel. Linear speedup in T would require bandwidth to grow with T , and on this machine it does not grow at all: a single core already draws 85.2% of what the module can supply, so there is no headroom for a second core to exploit, let alone an eighth.

Substantially improved: yes, but only by moving fewer bytes — and by a bounded factor. Section 8.7 achieves $1.283\times$, which is essentially all of the $1.333\times$ that is theoretically available.

The distinction matters. "Make it parallel" is the wrong question for this program; "make it move less data" is the right one. That is the general lesson of the roofline model, and it is the reason the same eight hardware threads that were worth $6.3\times$ in Program 3 are worth $0.92\times$ here.

It is worth stating what would actually fix this machine, since it is a configuration problem rather than a code problem: populating Channel B would double the theoretical peak to 42.7 GB/s. On a bandwidth-bound kernel running at 85.2% of a single channel, that second module is worth far more than any amount of rewriting — and, unlike the rewriting, it would also let the task version finally do something useful, since one core cannot saturate two channels.

8.6 Extra credit: why the multiplier is four, not three

`main.cpp` computes `TOTAL_BYTES = 4 * N * sizeof(float)` even though the kernel reads two floats and writes one. The missing fourth access is a read that the program never asked for.

Stores go through the cache. When `result[i]` is written, the hardware must first bring the containing 64-byte line into cache so that the parts of the line not being written are preserved, and so that the cache has a coherent copy to own. This is a *read-for-ownership*. Because `result` is 80 MB and the L3 is 8 MiB, every line misses, so every line is fetched from DRAM — and then immediately overwritten in full and later written back. The traffic per element is therefore one read of `X`, one read of `Y`, one read-for-ownership of `result`, and one writeback of `result`: four floats, of which the third is pure waste.

This explanation is testable, and two independent tests confirm it.

Test 1: remove the read-for-ownership and see if the time drops by the predicted amount. `_mm256_stream_ps` issues a non-temporal store, which writes a full line straight to memory without fetching it first. If the account above is right, traffic falls from $4N$ to $3N$ and the kernel should speed up by $4/3 = 1.333\times$. Measured: 17.734 ms $\rightarrow$ 13.825 ms, a factor of $1.283\times$, which is 96.2% of the prediction. A change that touches only how the results are written, and nothing about the arithmetic or the reads, recovered almost exactly one quarter of the runtime.

Test 2: a reported number that is physically impossible. `toBW` hard-codes the $4N$ assumption. Applied to the non-temporal variant it reports 21.556 GiB/s — above the 19.87 GiB/s that a single DDR4-2667 channel can physically transfer. No kernel can exceed its own memory bus, so the only possible conclusion is that this kernel is not moving $4N$ bytes. That is precisely the claim. The formula is right for the shipped code and wrong for the non-temporal version, which is the cleanest demonstration that the fourth N is the read-for-ownership and not something intrinsic to `saxpy`.

8.7 Extra credit: improving `saxpy`

The improvement follows from the analysis: since the kernel is bandwidth-bound at 85.2% of peak, the only lever is total bytes, and the only removable bytes are the read-for-ownership. Replacing the store with `_mm256_stream_ps` gives

17.734 ms $\rightarrow$ 13.825 ms, a **$1.283\times$ speedup, on one core**, beating all eight hardware threads of the shipped task version (19.150 ms) by $1.39\times$.

Two implementation details matter. Chunk boundaries are forced to whole 64-byte lines, because a non-temporal store covering part of a line still triggers the fetch it was meant to avoid, which would silently undo the optimisation. And `_mm_sfence` is issued at the end of each thread’s range, since non-temporal stores are weakly ordered with respect to everything else.

What the best possible implementation would achieve. With $3N$ floats of traffic and the 18.176 GB/s this machine has been observed to sustain, the floor is $240 \text{ MB} / 18.176 \text{ GB/s} = 13.2 \text{ ms}$. The measured 13.825 ms is within 4.7% of that floor, so there is essentially nothing left to win on this hardware — and the remaining gap is not worth chasing, because it is smaller than the run-to-run spread of the shipped program. Going faster requires a different memory configuration, not different code. On a dual-channel machine of the same generation the same source would be expected to run at roughly twice the speed, and only there would multi-threading begin to pay.

8.8 Summary

Question	Answer	Mechanism
Speedup from ISPC tasks?	$0.92\times$ (a slowdown)	Bus already saturated by 1 core
Is it granularity?	No, swept 1–256 tasks	Range only 14.3%, monotone
Is it ISPC’s runtime?	No, <code>std::thread</code> agrees	$0.90\times$ at 8 threads
Theoretical peak bandwidth	21.34 GB/s	Single-channel DDR4-2667
Achieved bandwidth	18.176 GB/s	85.2% of peak
FP units utilised	0.48% of 474 GFLOPS	$I = 0.125$ vs ridge 22.2
Near-linear speedup possible?	No	Bandwidth does not scale with T
Substantially improvable?	Yes, $1.283\times$	Move $3N$ instead of $4N$
Why the $4\times$ multiplier?	Read-for-ownership	Confirmed two ways

What this program demonstrates. Parallelism is not a universal speedup. Programs 1–4 were limited by how work was divided, and every one of them rewarded better division. `saxpy` is limited by a resource that division cannot multiply, so the same techniques that were worth $6\times$ elsewhere are worth less than nothing here. Recognising which regime a kernel is in — by comparing its arithmetic intensity against the machine’s ridge point, which takes no measurement at all — is what decides whether the next hour is better spent on threads or on data movement. This program is also the one place in the assignment where the machine’s own configuration, a single populated memory channel, sets the result more firmly than any code does.

9 Program 6: Making K-Means Faster

The starter code clusters $M = 1,000,000$ points of dimension $N = 100$ into $K = 3$ clusters, and it is already correct — the task is purely to find where the time goes and remove it. The handout deliberately does not say where to look. Two constraints shape everything that follows: only `kmeansThread.cpp` may be modified, and *only one* of `dist`, `computeAssignments`, `computeCentroids` and `computeCost` may be parallelised. Choosing which one is the whole problem, and that choice has to be made on evidence rather than intuition.

Dataset. All measurements use the supplied `data.dat`, 804,002,420 bytes, whose MD5 checksum is

3a25f24193f4fdca82ee4cb2737fd5bb

The file itself is not committed to the repository, on account of its size.

9.1 Step 1: measure before changing anything

`CycleTimer` probes were placed around the three phases inside the main loop, accumulating per-phase totals across the whole run and printing them when `KMEANS_PROFILE` is set. The run converges in 101 iterations.

One measurement precaution proved necessary. The first profiled run reported a total 11% above the clean baseline, and the excess was spread across phases that had not been touched. This is a 45 W mobile part running a single-threaded workload for roughly fifty seconds, which is long enough to heat it into a lower turbo bin, so a run taken immediately after another is measurably slower. The harness therefore profiles three times and keeps the fastest, consistent with the minimum-of- N rule used throughout this report. For the same reason the later comparisons are run as interleaved rounds — every configuration once per round, minimum taken across rounds — rather than all repetitions of one configuration before moving to the next, so that drift is shared between configurations instead of being charged to whichever one happens to run last.

Table 18: Where the 101 iterations spend their time, before and after. Times are totals over the whole run, from `CycleTimer` probes around each phase; the fastest of three profiling runs is reported.

Phase	Original		Final		Gain
	ms	Share	ms	Share	
<code>computeAssignments</code>	34409.5	68.3%	5960.1	27.1%	$5.77\times$
<code>computeCentroids</code>	5888.7	11.7%	5988.7	27.2%	$0.98\times$
<code>computeCost</code>	10076.6	20.0%	10054.4	45.7%	$1.00\times$
Total	50374.8		22003.2		$2.29\times$

`computeAssignments` accounts for 68.3% of the runtime. It is the only phase large enough for Amdahl’s Law to permit the required speedup, so it is the one function to parallelise.

The profile also explains *why* the phases cost what they do, which matters for knowing what can be recovered. `computeAssignments` performs $K \times M = 3 \times 10^6$ calls to `dist`, `computeCost` performs $M = 10^6$, and `computeCentroids` performs none — it streams $M \times N$ doubles and accumulates. The measured ratio of the first two phases is close to the 3 : 1 their call counts predict.

Why `dist` is slower than it looks. Disassembling the starter object file shows something worth knowing before trying to fix anything. GCC does fold `pow(x,2)` into a multiply — there is no call to `pow` in the generated code — and it does vectorise the subtract and square into `vsubpd/vmulpd`. But the accumulation comes out as a chain of scalar `vaddsd` instructions, because floating-point addition is not associative and the compiler may not reorder the sum without being told it can. Each `dist` call is therefore a dependency chain of $N = 100$ additions at roughly four cycles of latency each: about 400 cycles, and the arithmetic units sit idle between links. At 3×10^6 calls per iteration that predicts roughly 1.2×10^9 cycles, or 310 ms at the 3.881 GHz single-thread turbo recorded in Table 6. The measured cost is 340.7 ms per iteration — about 10% above the estimate, which is what one expects when the clock cannot hold peak turbo through a fifty-second run. A model this simple landing that close is good evidence that the dependency chain, and not memory or anything else, is what sets the cost.

This matters because it tells us the phase is *latency*-bound, not bandwidth-bound. Latency-bound work is exactly the kind that parallelism helps: eight threads run eight independent dependency chains at once.

9.2 Step 2: what Amdahl’s Law permits

With $f = 0.683$ of the runtime in the one function we are allowed to parallelise and $T = 8$ hardware contexts:

$$S_{max} = \frac{1}{(1-f) + \frac{f}{T}} = \frac{1}{0.317 + \frac{0.683}{8}} = \frac{1}{0.317 + 0.0854} = \frac{1}{0.4023} = 2.49\times.$$

Two things follow immediately. The handout’s $2.1\times$ target is achievable but not comfortably — it requires capturing $2.1/2.49 = 84.5\%$ of the theoretical maximum, so a merely adequate parallelisation will miss. And the 0.317 of runtime outside `computeAssignments` is untouchable under the one-function rule, which fixes the ceiling no matter how well the rest is done.

9.3 Step 3: the obvious parallelisation, and why it is the wrong one

The starter code hands `computeAssignments` a range `[args->start, args->end)` over the cluster index k , and its outer loop runs over exactly that range. The structure is an invitation to split on k , so that was tried first.

It requires more care than it appears to. Threads splitting on k all write the shared `minDist` and `clusterAssignments` arrays, so each thread needs private copies and a serial merge afterwards to pick the winning centroid per point. Implemented that way it is correct — it converges in the same 101 iterations — and it does speed the phase up.

Result: $1.68\times$ overall (28168.5 ms), short of the $2.1\times$ target.

The reason is structural rather than a defect of the implementation: $K = 3$. There are only three units of work on this axis, so at most three of the eight hardware contexts can ever be busy, and five sit idle. The phase itself sped up by $2.61\times$ — or $2.37\times$ once one allows for the two phases this variant does *not* touch having also run 10% faster in that run, so part of the apparent gain is clock rather than threads. Either way it is a reasonable return on three threads, but three threads is all the axis allows. Additionally, splitting on k keeps the original loop order, in which each thread streams the entire 800 MB data array for its own k — so total memory traffic is unchanged while the merge pass adds a new M -length scan.

The lesson is that the axis the code hands you is not necessarily the axis the problem has. $K = 3$ and $M = 10^6$ are both available; only one of them is larger than the machine.

9.4 Step 4: parallelising over data points

The fix inverts the loops. Instead of *for each centroid, scan every point*, the code does *for each point, compare against every centroid*:

```
for (int m = mStart; m < mEnd; m++) {
    const double *point = &args->data[(size_t)m * N];
    double best = 1e30; int bestK = -1;
    for (int k = args->start; k < args->end; k++) {
```

```

    double d = dist(point, &args->clusterCentroids[k * N], N);
    if (d < best) { best = d; bestK = k; }
}
args->clusterAssignments[m] = bestK;
}

```

This one change does three things at once.

1. **It creates a million units of work instead of three.** Each thread takes a contiguous range of m , so any thread count divides the work evenly and every thread does identical work — there is no load imbalance to manage, unlike Program 1.
2. **It removes all sharing.** Each point’s minimum is now a local variable and each thread writes only its own slice of `clusterAssignments`. The `minDist` array of M doubles disappears entirely, along with the merge pass that the k -split needed. No locks, no atomics, no false sharing beyond one cache line at each range boundary.
3. **It cuts memory traffic by a factor of K .** The original reads the whole 800 MB data array once per centroid; this reads each point once and compares it against all three centroids while it is still in registers. The three centroids are 3×100 doubles — 2.4 KB, permanently in L1. Given what Program 5 established about this machine’s memory system, this part is not incidental: the original ordering moves $K \times 800$ MB per iteration, which at the 18.176 GB/s measured in Program 5 would alone cost 132 ms.

Size of the change. The handout estimates its own solution at 20–25 lines, and the change that matters here is about 35: the point-major loop above, and the code that splits $[0, M)$ across threads. The file grows by more than that because the `CycleTimer` instrumentation from step 1 is kept, and because the original and the k -split implementations are retained behind an environment variable so that every comparison in this section can be reproduced from a single binary. Running `./kmeans` with nothing set selects the final implementation.

Result: $2.32\times$ overall (47378.1 ms $\rightarrow$ 20445.0 ms), against a handout target of $2.1\times$ and an Amdahl ceiling of $2.49\times$. That is 93.2% of the theoretical maximum.

Table 19: The three implementations, minimum of three runs each. All converge in 101 iterations, so all compute the same result.

Implementation	Threads used	Time (ms)	Speedup
Original (unmodified)	none	47378.1	1.00 $\times$
Parallel over cluster index k	$\min(T, K) = 3$	28168.5	1.68 $\times$
Parallel over point index m	$T = 8$	20445.0	2.32 $\times$
Amdahl bound S_{max} at $T = 8$			2.49 $\times$
Handout target			2.10 $\times$

Table 18 also carries a useful internal check. The two phases that were *not* modified, `computeCentroids` and `computeCost`, cost within 1.7% and 0.2% of their original values after the change. Nothing moved except what was meant to move, which both validates comparing the two profiles against each other and rules out the possibility that the speedup came from accidentally doing less work.

`computeAssignments` itself fell from 34409.5ms to 5960.1ms, a factor of $5.77\times$. That figure combines both changes — the loop interchange and the eight threads — so it should not be read as a parallel efficiency; Section 9.5 separates the two.

9.5 Step 5: confirming the bound is the binding constraint

Table 20: Thread sweep of the final implementation against the Amdahl bound computed from the measured $f = 0.683$. The $T = 1$ row exceeds the bound because the loop interchange is an improvement to the serial code, which Amdahl’s Law does not model.

Threads	Time (ms)	Measured speedup	Amdahl bound
1	38571.2	$1.23\times$	$1.00\times$
2	27650.4	$1.71\times$	$1.52\times$
3	23670.7	$2.00\times$	$1.84\times$
4	22031.8	$2.15\times$	$2.05\times$
6	21433.3	$2.21\times$	$2.32\times$
8	19767.1	$2.40\times$	$2.49\times$

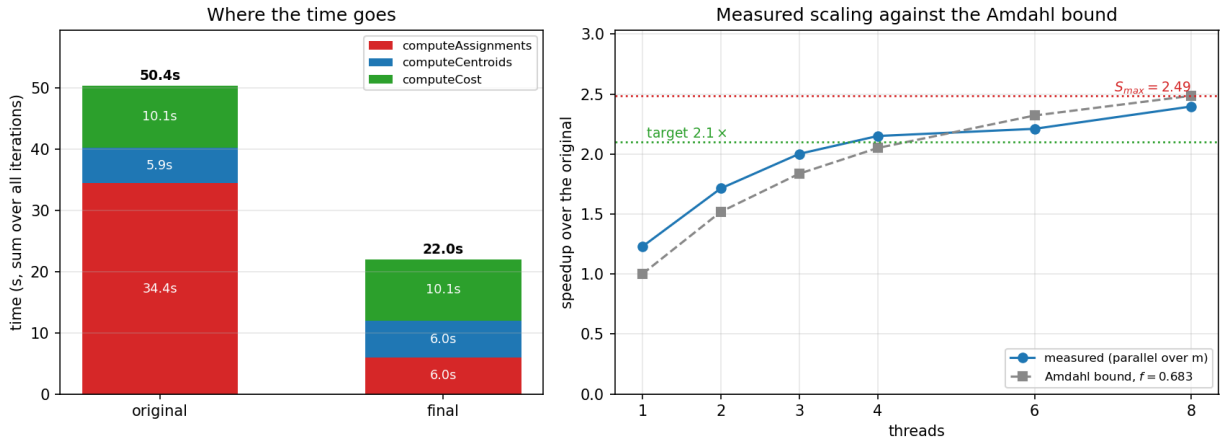


Figure 8: Left: where the time goes before and after; the two phases that were not modified are unchanged. Right: measured speedup against the Amdahl bound computed from the measured f . The two curves cross at about five threads — see the text for why.

The sweep confirms the diagnosis, and the shape of the comparison is more informative than a single number. Both curves flatten hard after four threads, because by then the parallel part has shrunk to the point where the fixed 0.317 of serial work dominates everything else.

The measured curve *crosses* the Amdahl bound at about five threads, which is the visible result of two opposing effects. Below the crossing it runs above the bound, which is not a contradiction: Amdahl’s Law bounds what parallelising a *fixed* amount of work can achieve, and the loop interchange reduced that work, so the measured curve is not bounded by a law computed from the original workload. Above the crossing it falls slightly below, because the threading does not achieve a perfect $1/T$ on the parallel portion — the same sub-linear scaling at high thread counts seen throughout this report, and for the same reasons: SMT contexts are not full cores, and turbo headroom is lost at full occupancy. The advantage from doing less work is roughly constant, while the shortfall from imperfect scaling grows with T , so one overtakes the other.

The $T = 1$ row separates the two contributions cleanly. With threading switched off entirely, the final code still runs in 38571.2ms against the original’s 47378.1ms — $1.23\times$ from the loop

interchange alone, before a single extra thread is involved. Threading the interchanged code then contributes a further $1.89\times$, and $1.23 \times 1.89 = 2.32$. Neither change would have reached the $2.1\times$ target by itself.

One inconsistency is worth stating rather than hiding: the sweep’s best eight-thread run, 19767.1 ms, is about 3% faster than the 20445.0 ms recorded in Table 19 for the same configuration. The two are separate experiments run at different times, and 3% is the run-to-run variation this workload shows on this machine. The headline figure quoted above is the slower of the two, because it comes from the experiment in which the original and the final implementation were measured in the same interleaved rounds and are therefore directly comparable.

The profile after optimisation (Table 18) shows the bottleneck has moved. `computeAssignments` has fallen to 27.1% and `computeCost` is now the largest phase at 45.7%. That is exactly where the remaining time is, and it is exactly what the rules forbid touching: `computeCost` is a second function, and the one parallelisation permitted has been spent. The only other route — reassociating the summation inside `dist` to break its dependency chain — is ruled out by the instruction not to change that function’s semantics. So $2.32\times$ is close to the constrained optimum, and the constraint, not the machine, is what now binds.

9.6 Correctness

Three independent checks, since a faster wrong answer is worthless:

- **Iteration count.** All three implementations converge in exactly 101 iterations. Convergence is decided by `stoppingConditionMet` comparing per-cluster costs against ϵ , so an identical count across a hundred iterations means the cost vector followed an identical trajectory.
- **No reordered arithmetic.** `dist` is called unmodified and each point’s comparisons happen in the same order as before, so the floating-point results are bit-identical rather than merely close. The parallelisation partitions m , and no partial sums cross threads.
- **Visual check.** `plot.py` output (Figure 9) shows three well-separated clusters with centroids at their centres, matching the reference images in the handout.

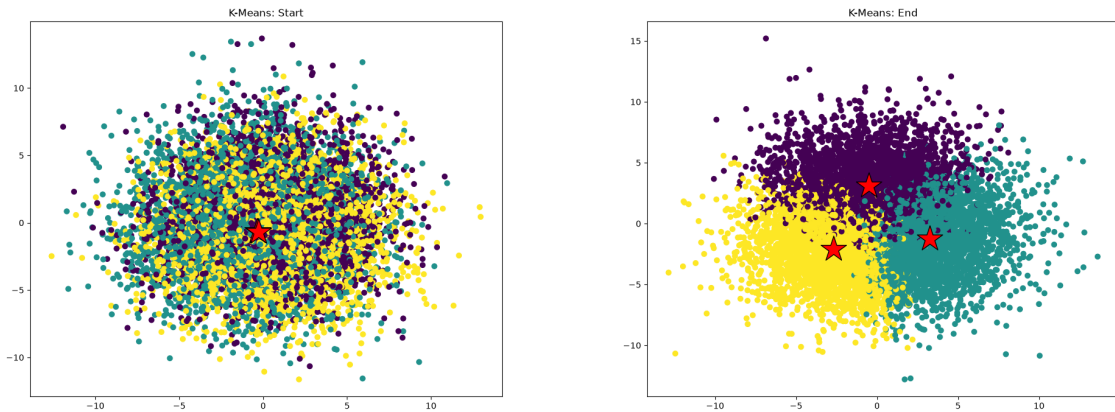


Figure 9: Cluster assignments before (left) and after (right) the algorithm runs, produced by `plot.py` from the final implementation. Red stars are centroids. The 100-dimensional points are projected to 2-D by PCA, so a few points appear nearer a centroid other than their own; the handout notes this is expected.

9.7 Summary

Question	Answer	Mechanism
Where is the time spent?	<code>computeAssignments</code> , 68.3%	$K \times M$ calls to <code>dist</code>
Why is that phase slow?	Latency, not bandwidth	100-long <code>vaddsd</code> chain
What does Amdahl permit?	$2.49\times$	$f = 0.683$, $T = 8$
First attempt: split on k ?	$1.68\times$, missed target	$K = 3 < T = 8$
Final: split on m ?	$2.32\times$	10^6 units, no sharing
Fraction of S_{max} ?	93.2%	—
Target met?	Yes, $2.1\times$ required	—
What binds now?	<code>computeCost</code> , 45.7%	Second function, not permitted

What this program demonstrates. The speedup came from one structural change, not from tuning: the original code was parallel along an axis of length three on a machine with eight contexts, and the fix was to notice that the same computation is also parallel along an axis of length a million. Profiling first is what made that visible — without the per-phase breakdown there would have been no reason to prefer `computeAssignments`, and no way to know that Amdahl’s Law left only $2.49\times$ on the table before any code was written. It is also the clearest illustration in this assignment that the bound moves: having removed the original bottleneck, the program is now limited by a phase the rules do not allow us to touch.